

Synatra

Smart Contract Security Assessment

September 2024

Prepared for:

Synatra

Prepared by:

Offside Labs

Allen Xu

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	4
4	Key Findings and Recommendations	5
4.1	Wrong Destination When Closing Account in fulfill_token IX	5
4.2	Sandwich Attack on Admin Rate Settings	6
4.3	Reinitialization on global.count may Overwrite Existing Pool	8
4.4	Possible Asset Loss Due to Unbounded stake_rate and unstake_rate	9
4.5	Centralization Risk in withdraw* and delete_account IX	10
4.6	Informational and Undetermined Issues	12
5	Disclaimer	13



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *yusd* smart contracts, starting on September 10, 2024, and concluding on September 11, 2024.

Project Overview

Synatra's Synthetic Staking is a platform designed to generate passive yield on the Solana blockchain. Currently, the platform offers two staking pools: ySOL and yUSD. Users can stake SOL and USD in these pools to earn rewards.

Synatra employs various arbitrage strategies to generate profits, including cash and carry arbitrage, hedged carry trade, and depeg arbitrage. The profits are distributed to users through a centralized mechanism that manages the stake rate and unstake rate for the pools.

Audit Scope

The assessment scope contains mainly the smart contracts of the *yusd* program for the *yusd* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- *yusd*
 - Branch: main
 - Commit Hash: 525e57b998134f2c88d0df6928bceff2a36717de
 - [Codebase Link](#)

We listed the files we have audited below:

- *yusd*
 - `programs/yusd/src/*.rs`

Findings

The security audit revealed:

- 0 critical issue
- 0 high issues
- 2 medium issues
- 3 low issues
- 2 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Wrong Destination When Closing Account in fulfill_token IX	Medium	Fixed
02	Sandwich Attack on Admin Rate Settings	Medium	Acknowledged
03	Reinitialization on global.count may Overwrite Existing Pool	Low	Fixed
04	Possible Asset Loss Due to Unbounded stake_rate and unstake_rate	Low	Fixed
05	Centralization Risk in withdraw* and delete_account IX	Low	Acknowledged
06	Unbounded ClaimRecord in fulfill_token and fulfill_sol IX	Informational	Fixed
07	Improper Account Closure in Some IXs	Informational	Fixed



4 Key Findings and Recommendations

4.1 Wrong Destination When Closing Account in fulfill_token IX

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

At the end of the `fulfill_token` IX, the ClaimRecord Account is closed, and the Rent associated with that account is withdrawn.

Typically, when an account is closed, the Rent is refunded to the original payer who covered it.

In this program, the Rent for the ClaimRecord Account is paid by the user who signs the unstake IX, and this is tracked in `ClaimRecord.claimer`.

In the `fulfill_sol` IX, the Rent is correctly refunded to the `ClaimRecord.claimer`. However, in `fulfill_token`, the Rent is mistakenly transferred to the IX signer of `fulfill_token`, who is usually not the `ClaimRecord.claimer`.

```
433 **ctx.accounts.signer.to_account_info().try_borrow_mut_lamports()? +=  
    claim_record_lamports;
```

[programs/yusd/src/lib.rs#L433](#)

Impact

Users who unstake non-SOL tokens cannot recover the rent they paid for creating the ClaimRecord account during the unstake process.

A more severe situation is, the `receipt_amount` recorded in a ClaimRecord is worth less than the account's rent. In this case, arbitrageur could exploit this by fulfilling the ClaimRecord before backend bot, spending only a small amount of tokens to gain the account's rent as profit.

Recommendation

Modify `ctx.accounts.signer` to `ctx.accounts.claimer`

```
**ctx.accounts.claimer.to_account_info().try_borrow_mut_lamports()? +=  
    claim_record_lamports;
```

Mitigation Review Log

Synatra Team: Commit c95ec5f17fac0dd109d7361fc4791f6e773814a3



The logic was intentionally written to prevent a different type of attack. i.e. it was done to prevent a malicious user from profiting from rent for ATA creations.

We did a different work around to resolve the issue, but it was intentionally made to return rent of the claim account to the fulfiller for SPL token based pools.

Offside Labs: **Fixed.**

The risk that the Synatra Team concerned about does indeed exist. From the project team's perspective, the original design was reasonable.

In their fix, the Synatra Team introduced a new design: rent is prepaid during `unstake` and then returned to users upon `fulfill`. This approach continues to mitigate the original risk while also returning the rent to users, effectively resolving the issue discussed here.

4.2 Sandwich Attack on Admin Rate Settings

Severity: Medium

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

Description

The `set_rates` IX allows the manager/oracle to update the `stake_rate` and `unstake_rate`. However, these transactions are vulnerable to a sandwich attack. The attacker can bundle their transactions in a way that stakes tokens right before the rate update, and then unstakes immediately after the new rates are applied.

This kind of attack works by exploiting the fact that the attacker is able to stake at the old, potentially more favorable `stake_rate`, and unstake at the newly set `unstake_rate`, effectively profiting from the rate difference without truly locking up their tokens. Since no real staking or holding period is required, this allows the attacker to perform risk-free arbitrage and drain value from the system.

Impact

This attack enables attackers to manipulate the staking mechanism by staking and unstaking tokens around the rate update, allowing them to profit from the rate changes. This could result in the attacker gaining an unfair advantage to drain funds from the staking system, negatively impacting the overall pool value and causing potential losses for other users.

Though the attack could only appear during manager/oracle's rate update, the potential financial loss could be significant if large amounts of tokens are involved.



Proof of Concept

An attacker can execute the following sequence:

1. Current rates config in pool `stake_rate (100)` , `unstake_rate (95)`
2. Pool manager submits a transaction to update the rates to: `stake_rate (110)` , `unstake_rate (105)` .
3. Attacker detects the admin's transaction and finds it is profitable.
4. Attacker sends a bundle of transactions:
 - First, stakes 10000 USDC at the current `stake_rate (100)` and receive 10000000 `receipt_token`.
 - The bundled rate updating transaction from Pool manager.
 - Unstaking all 10000000 `receipt_token` at the new `unstake_rate (105)` and waiting for backend to fulfill the ClaimRecord.
5. After fulfill_token, attacker receives 10500 USDC, with 500 USDC being the profit from this sandwich attack.

Attacker profits from the rate difference without holding the stake for any meaningful period.

Recommendation

It is recommended to introduce a **minimum staking cooldown period**. After staking tokens, users would be required to wait for a predefined period before they can unstake. This would ensure that no immediate arbitrage between rate changes is possible.

By implementing this cooldown, any staked tokens would need to remain locked for a certain period, making it impossible for attackers to exploit the rate changes instantly.

Mitigation Review Log

Synatra Team: Acknowledged.

Currently the way we mitigate mev arbs is to update rates through jito. We create a bundle that also arbs the LPs to get the prices consistent with the new rates.

Understanding that in the future if/when we open pool creation to the public, it can be an issue for less sophisticated admins who arent using jito, we will consider adding an optional cooldown setting for pools.

Offside Labs: Jito is currently safe. An optional cooldown setting should be added for pools before opening them to public creation.



4.3 Reinitialization on global.count may Overwrite Existing Pool

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `initialize` IX, the DEPLOYER/admin has the ability to modify the `global.count`. This variable tracks the number of created pools.

```
67 global.count = count;
```

[programs/yusd/src/lib.rs#L67](#)

However, since the IX lacks a check to ensure the count can only increase, it could inadvertently be updated to a smaller value due to misconfigurations.

If count is reduced, it may lead to the overwriting of existing pools. This issue could potentially result in serious data integrity problems, as new pools could overwrite old ones, leading to loss of funds or unexpected behavior in the contract.

Impact

If count is set to a value lower than the existing count, it could overwrite previously created pools. This could lead to a pool with the same ID being entirely different after being overwritten. Without admin involvement, users would not be able to withdraw the assets they staked in the original pool.

Although this issue could only be triggered with admin privileges, it still poses a risk of user funds loss and pool corruption if misused, either intentionally or accidentally.

Recommendation

Ensure that the new value of count is always greater than or equal to the current `global.count`. With this requirement, the count value can only increase over time, preventing the accidental or malicious overwriting of existing pools.

Mitigation Review Log

Synatra Team: Commit `a135ff7527bc9c1bdc39cb65929a6a4661aa0182`

Offside Labs: **Fixed.**



4.4 Possible Asset Loss Due to Unbounded `stake_rate` and `unstake_rate`

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Precision

Description

In the `create_pool` and `set_rates` IX, the `stake_rate` and `unstake_rate` parameters are not bounded by any upper limit and are stored into the pool account directly.

This could lead to issues in token calculations, particularly when the rates are set too high (e.g., greater than 100,000).

the current calculation logic for receipt tokens is as follows:

```
168 let receipt_amount = amount * 100000 / pool.stake_rate;
```

[programs/yusd/src/lib.rs#L168](https://github.com/offside-labs/yusd/src/lib.rs#L168)

When the rates exceed this threshold, users may stake tokens but receive zero receipt token in return due to integer truncation in the formula used for calculating receipt amounts.

Impact

Allowing unbounded `stake_rate` and `unstake_rate` could cause issues where users who stake tokens receive fewer or even zero receipt tokens in return.

While only the admin can create and configure pool parameters on current mainnet, if the `allow_public_pool_creation` flag is enabled in the future, anyone could create a pool. This opens the door for malicious pool creators to steal stakes from users.

Proof of Concept

The `stake_rate` of `pool-1` (USDC) on current mainnet is 109808.

A user stakes 1 USDC into the pool. The `receipt_amount` for this stake operation is calculated as: $1 * 100000 / 109808 = 0$.

As a result, the user receives nothing from this stake operation.

In a more extreme scenario, a pool creator sets up a pool with a `stake_rate` of 1,000,000,000,000,000.

A user stakes 1,000,000,000,000 USDC into the pool. The `receipt_amount` for this stake operation is calculated as: $1000000000000 * 100000 / 1000000000000000000 = 0$.

The user still can not receive any receipt token after this stake.



Recommendation

It is recommended to validate the upper limits of `stake_rate` and `unstake_rate` in `create_pool` and `set_rates`, ensuring that these limits are compatible with the formulas used in `stake/fulfill` to prevent precision issues during users' smaller amount operations.

Additionally, the same validation checks should also be added in `create_pool` as those used in `set_rates`.

```
require!(
    stake_rate * 9 / 10 <= unstake_rate && unstake_rate <=
    stake_rate,
    CustomError::InvalidRange
);
```

Mitigation Review Log

Synatra Team: Commit 74da1b6256f43735aa1b0d78390d929d588e64d5

Offside Labs: **Fixed.**

4.5 Centralization Risk in `withdraw*` and `delete_account` IX

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Centralization Risk

Description

The protocol currently depends on `DEPLOYER`, `global.admin`, `pool.manager` and `pool.oracle` as signers for many operation instructions. This setup introduces a centralization risk, where the exploitation of any one of these accounts could lead to asset loss in the protocol's pools.

The affected operation instructions include:

- `initialize`
- `create_pool`
- `withdraw_sol`
- `withdraw_token`
- `set_rates`
- `update_limit`
- `transfer_mint_authority`
- `delete_account`



Impact

Suppose the `DEPLOYER` user key is accidentally leaked, an attacker with the private key could:

1. Modify `global.admin` .
2. Enable or disable public pool creation, allowing or restricting anyone from creating pools.
3. Change the `mint_authority` for any pool's `receipt_token` .
4. Delete any PDA Account owned by the program and retrieve the Rent from these accounts.

By combining these actions, attacker could potentially extract all assets staked in any pool.

Suppose the `global.admin` user key is accidentally leaked, an attacker with the private key could:

1. Modify `global.admin` .
2. Enable or disable public pool creation, allowing or restricting anyone from creating pools.
3. Change the `mint_authority` for any pool's `receipt_token` .

By combining these actions, attacker could potentially extract all assets staked in any pool.

Suppose the `pool.manager` user key is accidentally leaked, an attacker with the private key could:

1. Withdraw all SOL/tokens from the pool.
2. Manipulate the `stake_rate` / `unstake_rate` within the pool to control prices as desired.
3. Adjust the `receipt_token` limit to influence the pool's total supply.

By combining these actions, attacker could extract assets staked in a specific pool.

Suppose the `pool.oracle` user key is accidentally leaked, an attacker with the private key could manipulate the `stake_rate` / `unstake_rate` within the pool. The price can be arbitrarily controlled, allowing for the withdrawal of staked assets from the pool.

Mitigation Review Log

Synatra Team: Acknowledged.

Yes, Introducing multisig for updates to our stake rates or limits would help reduce single points of failure. our pools could/should use multisig updates.

However future pools may or may not utilize multisig since it's based on the entity that controls that particular pool.

We will make an update for multisig option on pools.



4.6 Informational and Undetermined Issues

Unbounded ClaimRecord in fulfill_token and fulfill_sol IX

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

In the `fulfill_token` and `fulfill_sol` IX, the `claim_record` is not bound to the pool. Instead, the claim record is passed directly as an account, and only its fulfilled flag is checked.

It's recommended to add a new check `claim_record.pool_id == pool.id`, to bind the pool to the `claim_record`.

Synatra Team: Commit 1b8cab8abe6f1f6b67a98cad52656511ec6222fa

Offside Labs: Fixed

Improper Account Closure in Some IXs

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

In the `fulfill_token` and `fulfill_sol` instructions, the program does not correctly close the `claim_record` account after it has been fulfilled. While there is an attempt to transfer the rent out from the `claim_record`, the account is not properly closed. The account data is not cleared, and there is no identifier to mark it as closed. These accounts may be vulnerable to revival attacks, where an attacker could transfer rent to those accounts after they have been closed in the same transaction.

The same issue also exists in the `delete_account` instruction.

It's recommended to close accounts using the `Anchor close constraint`.

Synatra Team: Commit 33835d6047886db290cf032869fb771bc34614b9

Offside Labs: Fixed



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

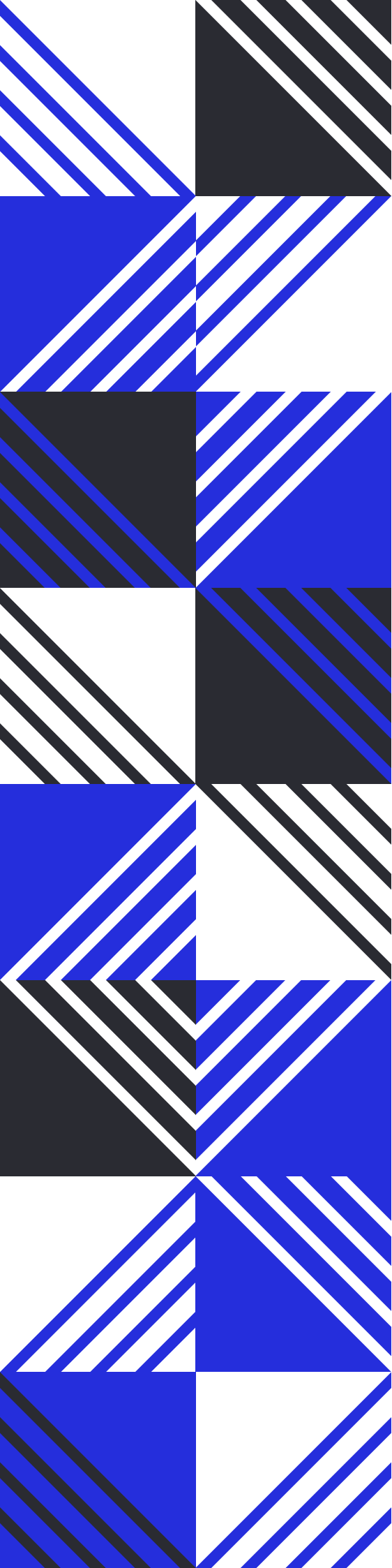
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs